[2026-1] AI프로그래밍

4주차-1: 파이썬 디버깅

2026.03.25 (수)

김현준

개념

__name__ 과 __main__ 이란?

- 파이썬 파일을 실행하면 `__name__` 이라는 특수 변수가 자동으로 생성
- 직접 실행할 경우: `__name__ == '__main__'`
- 다른 파일에서 import할 경우: `__name__ == '파일명(모듈명)'`

```
# 직접 실행 시
print(__name__)
# 출력: __main__

# import 시 (위 내용을 mymodule.py에 넣어둠)
import mymodule
# 출력: mymodule
```

개념

main() 함수와 if __name__ == '__main__'

- main() 함수: 프로그램의 진입점(entry point)을 명시적으로 정의하는 관례
- if __name__ == '__main__': 블록 안의 코드는 **"직접"** 실행할 때만 동작
- import로 불러올 경우 해당 블록은 실행되지 않음
- 코드 재사용성과 모듈화를 위한 파이썬의 핵심 패턴

```
def main():
    print("프로그램 시작!")
    result = my_function()
    print("결과:", result)

def my_function():
    print(str(42))
    return 42

if __name__ == '__main__':
    main()  # 직접 실행 시에만 호출됨
```

개념

import vs 직접 실행 – 차이 비교

🟢 직접 실행 (python test.py)

```
print(__name__)  # __main__
if __name__ == '__main__':
    print("MAIN")  # 실행됨
```

- __name__ 값: '__main__'
- if 블록: 실행됨
- greet() 호출됨

🔵 import 할 때 (import test)

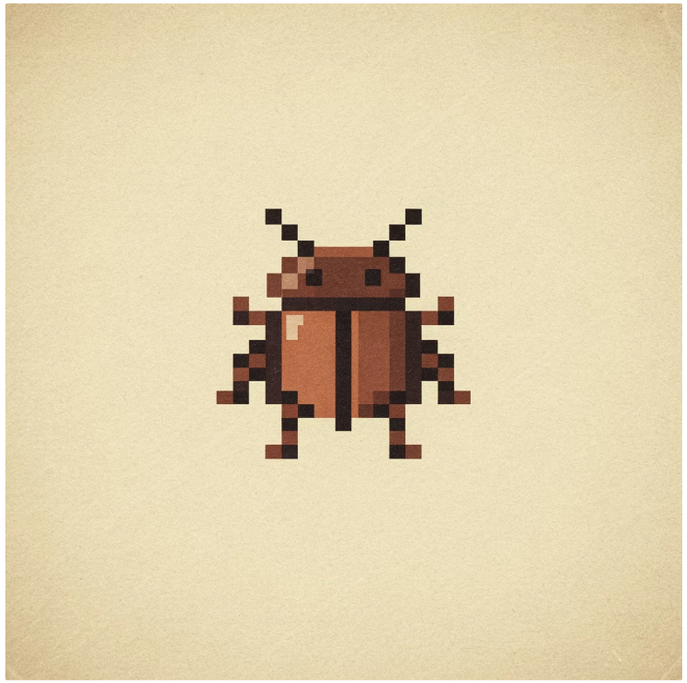
```
import test
# test 내부 __name__ = 'test'
# 내부 main함수 실행 X
test.my_function()  # 함수는 사용 가능
```

- ttest.py 내부 __name__ 값: 'test' (모듈명)
- main 함수는 if문 만족 X로 인해 실행 X
- 함수 정의만 불러와짐

핵심: import 시 불필요한 코드 실행을 막기 위해 if __name__ == '__main__': 활용

개념

디버깅(Debugging)이란?



- 버그(bug): 과거 에니악(진공관 컴퓨터)에 벌레가 끼어 타 죽으면서 코드에 오류가 발생한 것이 유래
- 디버깅(debugging): 버그를 해결하는 것
- = 내 의도대로 코드가 오류 없이 동작하게 만드는 행위
- Workflow: 라이브러리를 가져와서 내 목적에 맞게 코딩 → 테스트 → 디버깅

코딩

테스트

디버깅

개념

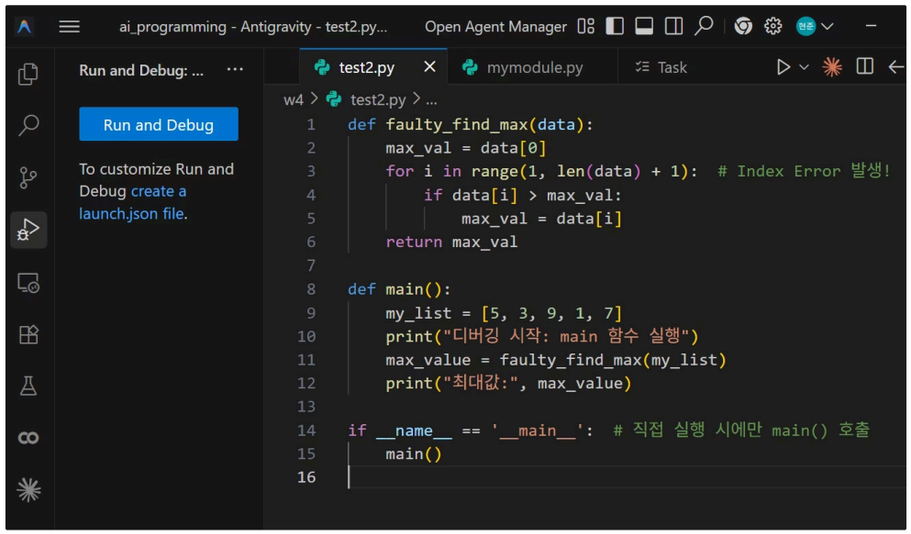
예시 코드 – 버그가 있는 코드

```
def faulty_find_max(data):
    max_val = data[0]
    for i in range(1, len(data) + 1):  # Index Error 발생!
        if data[i] > max_val:
            max_val = data[i]
    return max_val

def main():
    my_list = [5, 3, 9, 1, 7]
    print("디버깅 시작: main 함수 실행")
    max_value = faulty_find_max(my_list)
    print("최대값:", max_value)

if __name__ == '__main__':  # 직접 실행 시에만 main() 호출
    main()
```

- - range(1, len(data) + 1) → 마지막 인덱스가 len(data)=5가 되어 IndexError 발생
- - if __name__ == '__main__': 패턴으로 직접 실행 시에만 main() 호출

실습

Antigravity/VS Code 디버깅 시작하기



1. 좌측 사이드바에서 벌레+재생 아이콘 클릭 (Run and Debug)
2. 단축키: Ctrl + Shift + D
3. create a launch.json file.
4. Python Debugger 선택
5. Python File 선택
6. 파이썬 파일이 켜진 상태에서 "Run and Debug" 또는 F5 키

실습

launch.json

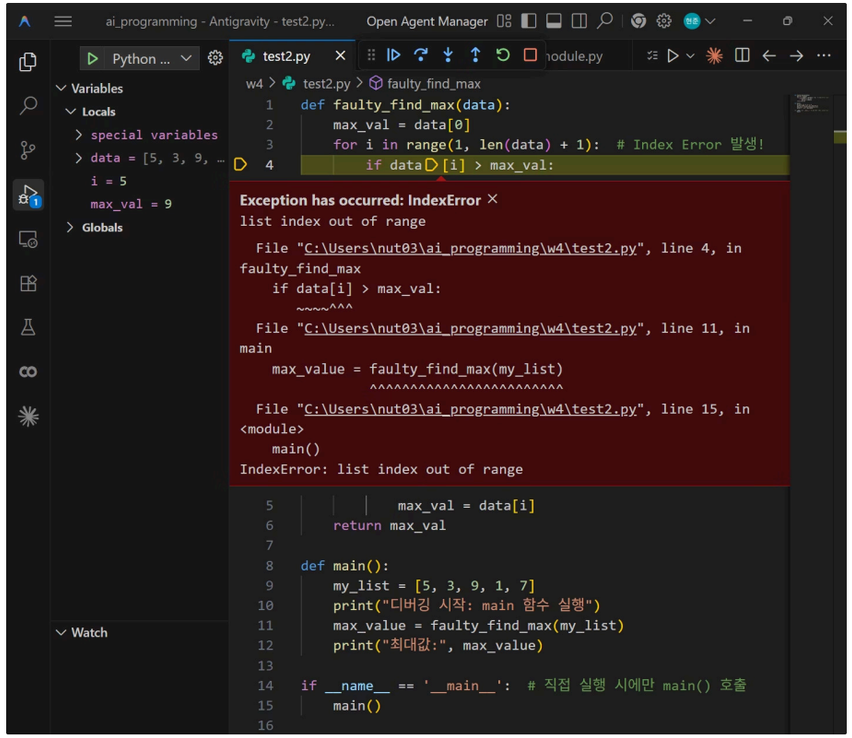
```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Python Debugger: Current File",
      "type": "debugpy",
      "request": "launch",
      "program": "${file}",
      "console": "integratedTerminal"
    }
  ]
}
```

launch.json에서 디버깅 환경을 세부 설정할 수 있음 (인자 전달, 환경변수 등)

https://code.visualstudio.com/docs/python/debugging#_set-configuration-options

개념

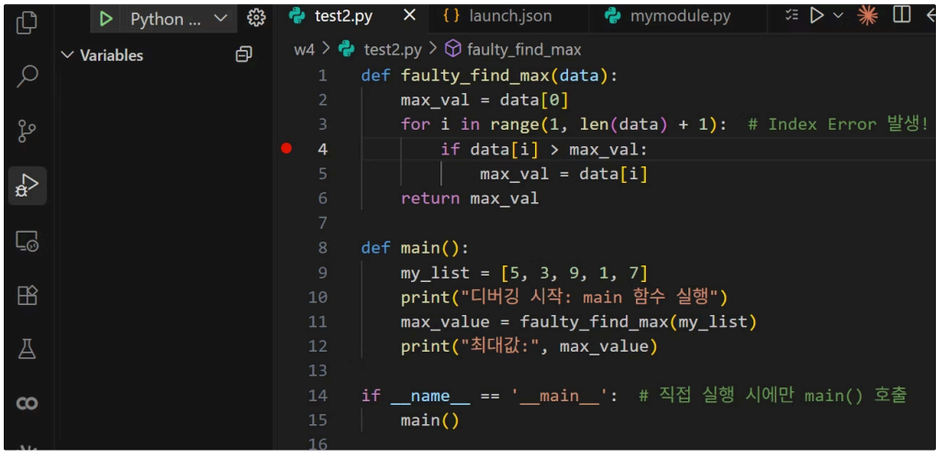
에러 발생 시 – 디버거가 멈추는 곳



- 에러가 발생한 줄에서 자동으로 멈춤
- 에러 팝업에서 확인할 수 있는 정보:
  - 어떤 에러 유형인지 (예: IndexError)
  - 에러 메시지 (예: list index out of range)
  - 어느 파일, 몇 번째 줄에서 발생했는지
  - 함수 호출 스택(call stack) 전체

개념

브레이크포인트(Breakpoint) 설정

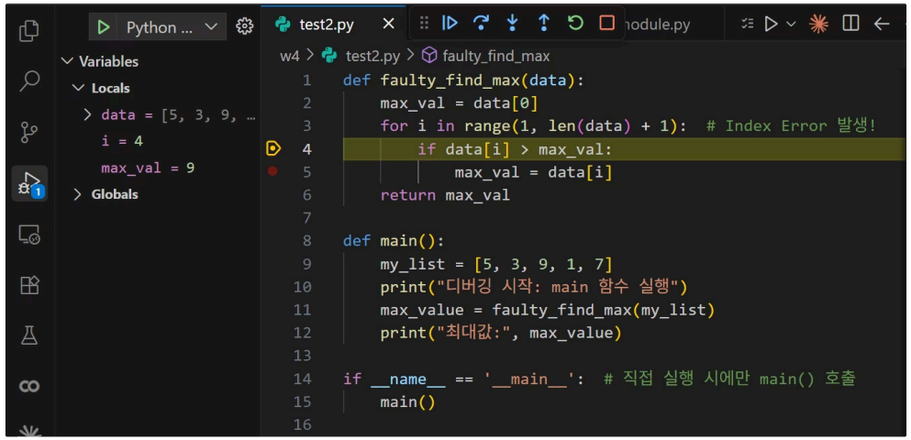


- 라인 번호 왼쪽을 클릭하면 빨간색 점(●)이 생김
- 디버깅 시 해당 위치 실행 "**직전**"에서 잠시 멈춤
- 멈춘 상태에서 변수값 확인 가능
- 우클릭 → 추가 설정 가능:
  - Edit Breakpoint...
  - 조건부 브레이크포인트: 특정 조건이 만족될 때만 멈춤
    - Expression: 특정 조건
      - i == 4 일 때만
    - Hit count: n번째 실행 시에만 멈추기

💡 브레이크포인트는 여러 개 설정 가능! 원하는 위치마다 클릭해서 추가하자

개념

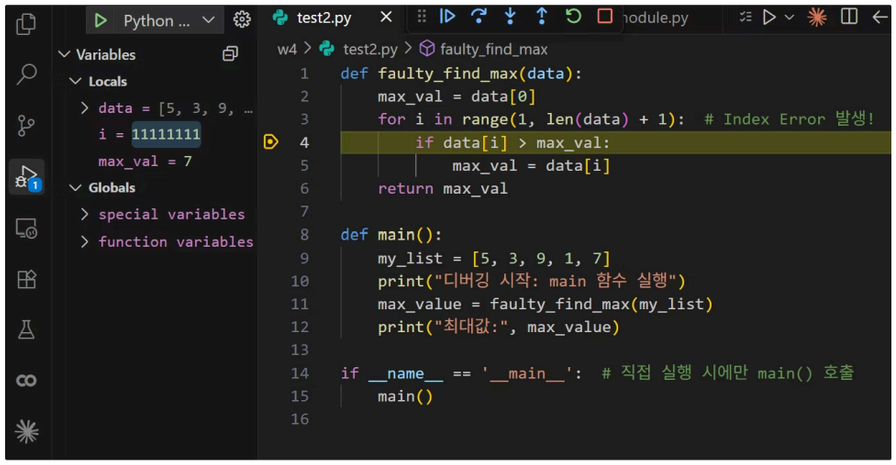
디버깅 중 변수값 확인 (Variables 창)



- 좌측 Variables 창에서 현재 위치의 모든 변수값 확인 가능
- Locals: 현재 함수 내 지역 변수
- Globals: 전역 변수
- 변수 옆 화살표(▶️)를 클릭하면 리스트/딕셔너리 내부도 펼쳐볼 수 있음
- 코드 위에 마우스를 올려도 변수값이 툴팁으로 표시됨

개념

디버깅 중 변수값 변경



- Variables 창에서 변수를 더블 클릭하면 값을 임시로 변경 가능
- 변경된 값으로 이후 코드가 실행됨
- 버그 수정 없이 다른 값으로 테스트해볼 때 유용

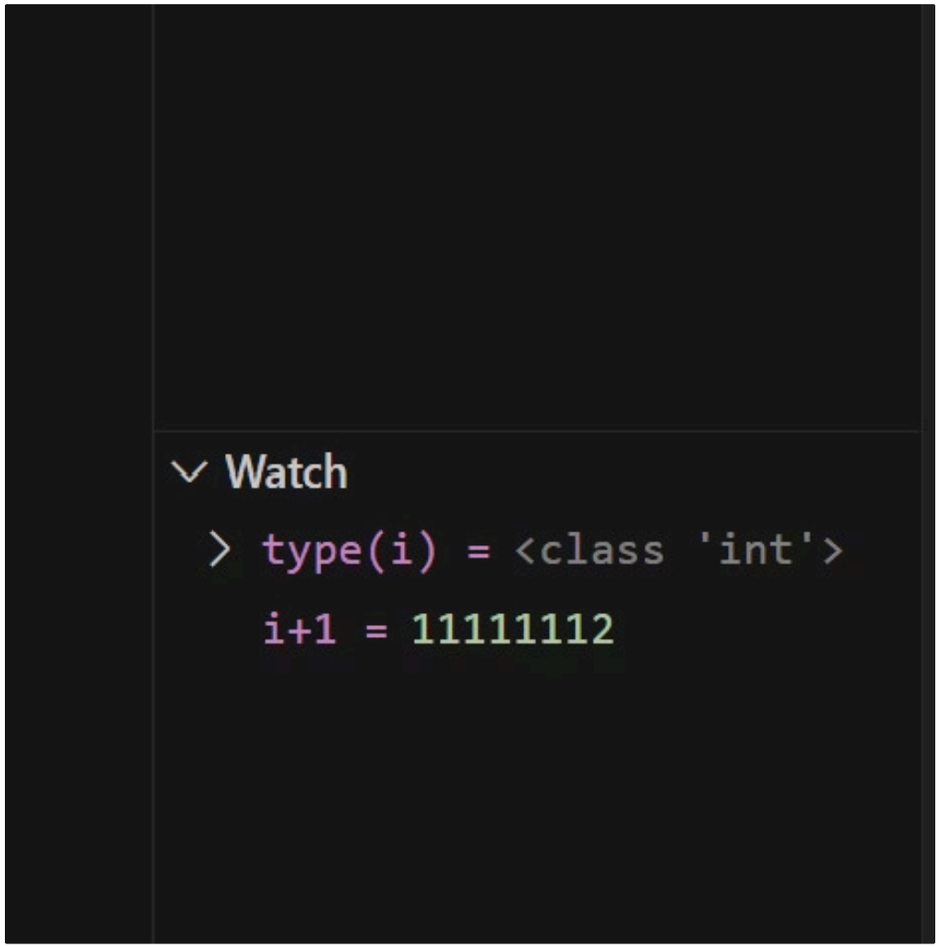
Example:

i = 4 → 더블클릭 → 11111111으로 변경 → 이후 코드는 i=11111111 가 된 상태로 계속 실행됨

⚠️ 주의: 변수값 변경은 프로그램 실행에 직접 영향을 줌! 의도치 않은 동작이 발생할 수 있으므로 신중하게 사용할 것

개념

디버깅 중 Watch – 감시 표현식



- WATCH 창: 내가 원하는 변수나 표현식을 등록해서 실시간으로 값 관찰
- 추가 방법: + 버튼 클릭 또는 빈 공간 더블 클릭
- 단순 변수뿐 아니라 계산식도 등록 가능

```
max_val        → 6
type(max_val)  → <class 'int'>
len(data)      → 5
data[i]        → 현재 i번째 값
```

⚠️ Watch에서 함수를 **실행**하면 프로그램 실행에 영향을 줄 수 있으므로 주의!

개념

한 단계씩 코드 진행 – 디버그 버튼

▶️ Continue (F5)

다음 브레이크포인트까지 계속 실행

⤵ Step Over (F10)

현재 줄 실행 (함수 내부로 들어가지 않음)

↓ Step Into (F11)

함수 내부로 들어가서 한 줄씩 실행

↑ Step Out (Shift+F11)

현재 함수에서 빠져나와 호출한 곳으로 복귀

⟳ Restart / ■ Stop

Ctrl+Shift+F5 재시작 / Shift+F5 중단

💡 Step Over와 Step Into의 차이: 함수를 만났을 때 Over는 건너뛰고, Into는 함수 안으로 들어간다

실습

버그 수정 – IndexError 해결하기

- 에러 분석: i = 5, len(data) = 5
- data[5]는 존재하지 않음 → IndexError 발생
- 원인: range(1, len(data) + 1) → 마지막 i가 5가 됨
- 수정: len(data) + 1 을 len(data) 로 변경

```
for i in range(1, len(data)):  # 수정 완료
```

수정 후 F5로 다시 실행하면 정상적으로 최대값 9가 출력됨

실습

실습 2 – 버그 있는 코드 디버깅하기

```
def faulty_build_list(data):
    total = 0
    result = ()
    for i, item in enumerate(data):
        total = i + item
        result.append(total)
    return result

def main():
    sample_data = [10, 20, 30, 40]
    print("main 함수 시작")
    built_result = faulty_build_list(sample_data)
    print("최종 결과:", built_result)

if __name__ == '__main__':
    main()
```

목표: 버그를 해결해서 현재 index까지 더한 값들로 구성된 tuple을 리턴하세요

예상 출력: 최종 결과: (10, 21, 32, 43)

실습

실습 2 정답 – 버그 3가지 수정

```
def faulty_build_list(data):
    total = 0
    result = []         # 수정 1: list로 변경
    for i, item in enumerate(data):
        total = i + item
        result.append(total)
    result = tuple(result)  # 수정 2: 마지막에 tuple 변환
    return result

def main():
    sample_data = [10, 20, 30, 40]
    print("main 함수 시작")
    built_result = faulty_build_list(sample_data)
    print("최종 결과:", built_result)

if __name__ == '__main__':
    main()
```

- 수정 1: result = () → result = [] (list는 append 가능)
- 수정2: 마지막에 tuple로 변환

```
main 함수 시작
최종 결과: (10, 21, 32, 43)
```

정리

오늘 배운 것 정리

__name__ & __main__

직접 실행 시 '__main__', import 시 모듈명

main() 패턴

if __name__ == '__main__': 으로 진입점 명시

디버깅 기본

브레이크포인트 설정 → F5 실행 → 에러 위치 확인

Variables & Watch

변수값 실시간 확인 및 감시 표현식 등록

단계별 실행

F10 Step Over / F11 Step Into / Shift+F11 Step Out